

RICORSIONE

Forma di programmazione alternativa
all'iterazione

Funzione/procedura ricorsiva $\rightarrow$ Una funzione
si dice ricorsiva se
richiama se stessa

```
int fatt (int m) {  
    if (m <= 1)  
        return 1;  
    else  
        return m * fatt(m-1);  
}
```

} informatico

$\nwarrow$ non è una chiamata in coda

$\left\{ \begin{array}{l} \text{fatt}(1) = \text{fatt}(0) = 1 \\ \text{fatt}(m) = m * \text{fatt}(m-1) \end{array} \right.$

} matematico

fib(1) = fib(1)

NO Induttivo

ricorsivo $\searrow$

```
int fib (int m) {  
    if (m=1)  
        return fib(1)  
}
```

RICORSIONE

Forma di programmazione alternativa
all'iterazione

Funzione/procedura ricorsiva → Una funzione
si dice ricorsiva se
richiama se stessa

```
int fatt (int m) {  
    if (m <= 1)  
        return 1;  
    else  
        return m * fatt(m-1);  
}
```

} informatico

non è una chiamata
in code

$\begin{cases} \text{fatt}(1) = \text{fatt}(0) = 1 \\ \text{fatt}(m) = m * \text{fatt}(m-1) \end{cases}$

} matematico

fib(1) = fib(1)

NO induttivo

ricorsivo ↘

```
int fib (int m) {  
    if (m=1)  
        return fib(1)  
}
```

$$\begin{cases} f_{00}(0) = 0 \\ f_{00}(m) = f_{00}(m+1) \end{cases}$$

no induttivo

```
int ff00(int m) {
  if (m == 0) return 0;
  else return gf00(m+1);
}
```

Ricorsivo

Perché un linguaggio permetta la ricorsione

↳ Procedure nel linguaggio

↳ Gestione dinamica della memoria

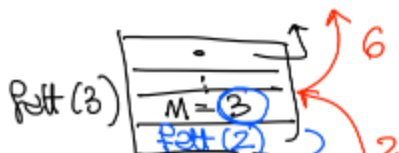
Ricorsione in code → forma ottimizzata (rispetto all'uso della memoria) della ricorsione

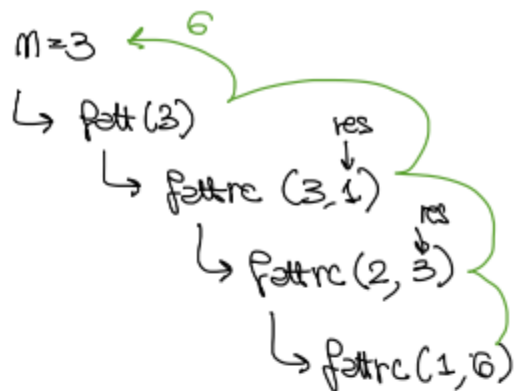
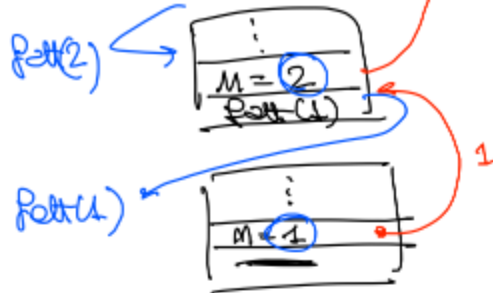
Una chiamata ^f ad una procedura ^g si dice in code se f restituisce il valore ritornato da g senza ulteriori computazioni.

Una procedura si dice ricorsiva in code se la chiamata ricorsiva è in code

```
int fatt(m) {
  return fattrec(m, 1);
}
```

```
int fattrec(int m, int res) {
  if (m <= 1) return res;
  else return fattrec(m-1, res * m);
}
```





$$\text{Fib}(1) = 1$$

$$\text{Fib}(2) = 1$$

$$\text{Fib}(m) = \text{Fib}(m-1) + \text{Fib}(m-2)$$

↑

```
int fib (int m) {
```

```
    if m <= 1 return 1 ;
```

```
    else return fib(m-1) + fib(m-2);
```

~~Now E~~ IN CODE

```
}
```

↘ in code

```
int fib(m) { return fibrec(m, 1, 1); }
```

$\text{fib}(m-2)$ $\text{fib}(m-1)$
 ↓ ↓

```
int fibrec (int m, int res1, int res2) {
```

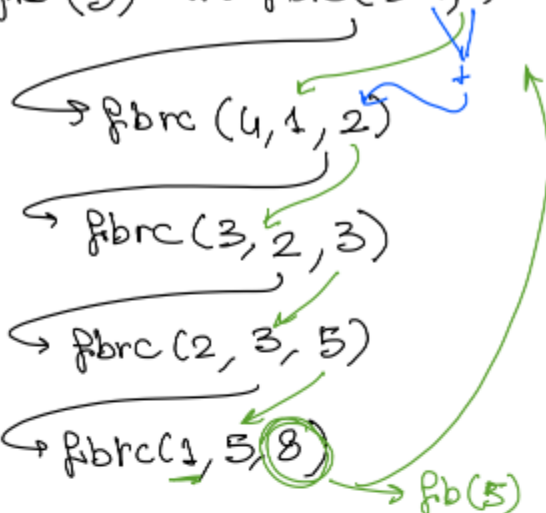
```
    if (m <= 1) return res2;
```

```
    else return fibrec (m-1, res2, res1 + res2);
```

↑

```
}
```

fib(5) $\leadsto$ fibrec(5, 1, 1)



TIPLOGIE DI ESERCIZI

Severazione di un comando

Scoping statico/dinamico

Calcolo statico + CFI

Completamento codice

Scoping + binding

Ricorsione in coda

Passaggio di parametri

Induzione strutturale

Esercizio di scoping di completamento del codice

```

{ int i;
  ⑧
  for (int i = 0; i <= 1; i++) {
    int x;
    x = fun(i);
  }
  
```

Fornire il codice da inserire in ④ in modo tale che

1. Se il linguaggio ha scoping statico le due chiamate a fun restituiscono lo stesso valore
2. Se il ling. ha scoping dinamico le due chiamate a fun restituiscono valori diversi.

→ il codice deve essere eseguibile → dobbiamo definire fun()

→ Perché il comportam. di un programma sia potenzialmente diverso in dipendenza delle regole di scoping

è NECESSARIO che ci sia

→ una procedura con ambiente non locale

→ il riferimento non locale deve esistere sia nel contesto di definizione che in quello di chiamata

→ i è un buon candidato per essere ambiente non locale per fun

è non è inizializzato nell'ambiente di chiamata
→ non si può usare

④ $\left\{ \begin{array}{l} i = 1; \\ \text{int fun}() \end{array} \right. \leftarrow$ serve per rendere il codice eseguibile
 $\left\{ \begin{array}{l} \text{return } i; \end{array} \right.$

Esecuzione con scoping statico

→ la chiamata di fun() restituisce lo i

non viene modificata
⇒ restituisce 1 ad
entrambe le chiamate

Esecuzione con scoping dinamico

↳ le due chiamate di `fun()` fanno
un riferimento alla `i` del scope for che
cambia tra una chiamata e l'altra
Prima chiamata → return 0
Seconda chiamata → return 1

Esercizio 2

```
int x;
```

⊗

```
void exec() {
```

```
    for (int j = 2; j <= 3; j++) {
```

```
        int y = 1;
```

```
        x = fun();
```

```
    }
```

```
exec();
```

```
}
```

codice t.c. `fun()` restituisce lo
stesso valore nelle due
chiamate con scoping
statico;

restituisce
valori diversi
con scoping
dinamico

→ Dobbiamo definire `fun()` con ambiente
non statico che possa dipendere
dalle regole di scoping.

↳ uniche variabili candidate
sono `y` e `j` perché l'ambiente
di chiamata non è modificabile
tra le due `j` cambia ad ogni
esecuzione quindi permetterebbe
di rispettare la richiesta.

→ Dobbiamo definire anche una j globale che non viene modificata.

⇒ $\textcircled{*}$ $\begin{cases} \text{int } j = 1 \\ \text{int fun}() \{ \text{return } j; \} \end{cases}$

Scoping
statico

La chiamata ad
ogni iterazione restituisce
la j globale che
vale 1

Scoping
dinamico

La chiamata restituisce
la j del ciclo for che
vale 2 alla prima iterazione
e 3 alla seconda.

Esercizio 3

Inizializzo

come nel primo esercizio
che il blocco del
while posso definire

un ambiente locale
separato da quello globale

int a = 0;

$\textcircled{*}$ while (a <= 1) {

int x;

$\textcircled{**}$ x = fun();

a++;

}

richiesta
come
per esercizio
precedente

Definiamo fun() per renderlo eseguibile
e fun() deve contenere un ambiente non
locale per dipendere dallo scoping.

$\textcircled{*}$ $\begin{cases} \text{int } x = 2; \\ \text{int fun}() \{ \text{return } x; \} \end{cases}$

scoping statico

L'ambiente non
locale per fun è
quello globale
quindi le due
chiamate restituiscono
2 (valore di x
non modificato)

Scoping dinamico

x non è
invariante

la prima chiamata restituisce $x=a=0$

la seconda chiamata restituisce $x=a=1$

For:

```
{ int a=0; int y=1;
```

⊗

```
void exec() { int x;
```

⊗

```
    y = fun();
```

```
    a++;
```

```
    while (a <= 1) { exec(); }
```

```
}
```

richieste
esercizio
precedenti